

Numerical Recipes Software 2007, "Dense Output for Stoermer's Rule," *Numerical Recipes Webnote No. 22*, at <http://www.nr.com/webnotes?22> [1]

17.5 Stiff Sets of Equations

As soon as one deals with more than one first-order differential equation, the possibility of a *stiff* set of equations arises. Stiffness typically occurs in a problem where there are two or more very different scales of the independent variable on which the dependent variables are changing. For example, consider the following set of equations [1]:

$$\begin{aligned} u' &= 998u + 1998v \\ v' &= -999u - 1999v \end{aligned} \quad (17.5.1)$$

with boundary conditions

$$u(0) = 1 \quad v(0) = 0 \quad (17.5.2)$$

By means of the transformation

$$u = 2y - z \quad v = -y + z \quad (17.5.3)$$

we find the solution

$$\begin{aligned} u &= 2e^{-x} - e^{-1000x} \\ v &= -e^{-x} + e^{-1000x} \end{aligned} \quad (17.5.4)$$

If we integrated the system (17.5.1) with any of the methods given so far in this chapter, the presence of the e^{-1000x} term would require a stepsize $h \ll 1/1000$ for the method to be stable (the reason for this is explained below). This is so even though the e^{-1000x} term is completely negligible in determining the values of u and v as soon as one is away from the origin (see Figure 17.5.1).

This is the generic disease of stiff equations: We are required to follow the variation in the solution on the shortest length scale to maintain the stability of the integration, even though accuracy requirements allow a much larger stepsize.

To see how we might cure this problem, consider the single equation

$$y' = -cy \quad (17.5.5)$$

where $c > 0$ is a constant. The explicit (or *forward*) Euler scheme for integrating this equation with stepsize h is

$$y_{n+1} = y_n + hy'_n = (1 - ch)y_n \quad (17.5.6)$$

The method is called explicit because the new value y_{n+1} is given explicitly in terms of the old value y_n . Clearly the method is unstable if $h > 2/c$, for then $|y_n| \rightarrow \infty$ as $n \rightarrow \infty$ even though the solution of (17.5.5) is bounded.

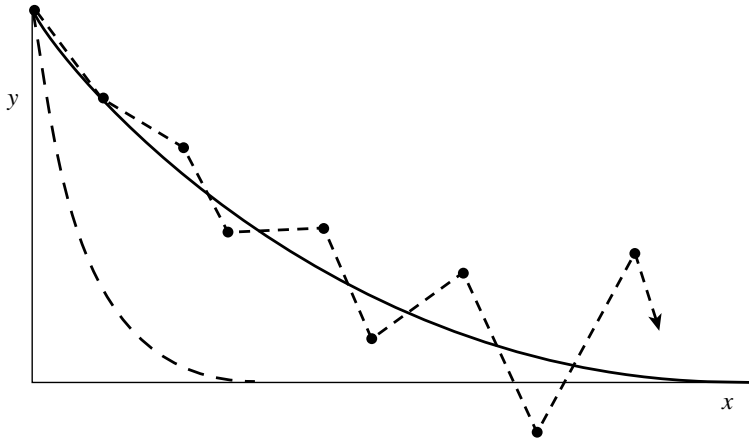


Figure 17.5.1. Example of an instability encountered in integrating a stiff equation (schematic). Here it is supposed that the equation has two solutions, shown as solid and dashed lines. Although the initial conditions are such as to give the solid solution, the stability of the integration (shown as the unstable dotted sequence of segments) is determined by the more rapidly varying dashed solution, even after that solution has effectively died away to zero. Implicit integration methods are the cure.

The simplest cure is to resort to *implicit* differencing, where the right-hand side is evaluated at the *new* y location. In this case, we get the *backward Euler* scheme:

$$y_{n+1} = y_n + hy'_{n+1} \quad (17.5.7)$$

or

$$y_{n+1} = \frac{y_n}{1 + ch} \quad (17.5.8)$$

The method is absolutely stable: Even as $h \rightarrow \infty$, $y_{n+1} \rightarrow 0$, which is in fact the correct solution of the differential equation. If we think of x as representing time, then the implicit method converges to the true equilibrium solution (i.e., the solution at late times) for large stepsizes. This nice feature of implicit methods holds only for linear systems, but even in the general case implicit methods give better stability. Of course, we give up *accuracy* in following the evolution toward equilibrium if we use large stepsizes, but we maintain *stability*.

These considerations can easily be generalized to sets of linear equations with constant coefficients:

$$\mathbf{y}' = -\mathbf{C} \cdot \mathbf{y} \quad (17.5.9)$$

Consider first the usual case where the matrix $\mathbf{C}$ can be diagonalized by a similarity transformation (cf. eqn. 11.0.11)

$$\mathbf{T} \cdot \mathbf{C} \cdot \mathbf{T}^{-1} = \text{diag}(\lambda_0 \dots \lambda_{N-1}) \quad (17.5.10)$$

where λ_i are the eigenvalues of $\mathbf{C}$. If we define the vector $\mathbf{z}(x)$ by $\mathbf{z} = \mathbf{T}^{-1} \cdot \mathbf{y}(x)$, then equation (17.5.9) becomes

$$\mathbf{z}' = -\text{diag}(\lambda_0 \dots \lambda_{N-1}) \cdot \mathbf{z} \quad (17.5.11)$$

This is a set of N independent equations for the components of $\mathbf{z}$ with solution

$$\mathbf{z} = \text{diag}(e^{-\lambda_0 x} \dots e^{-\lambda_{N-1} x}) \cdot \mathbf{z}_0 \quad (17.5.12)$$

Thus the solution of the original equation is

$$\mathbf{y} = \mathbf{T} \cdot \text{diag}(e^{-\lambda_0 x} \dots e^{-\lambda_{N-1} x}) \cdot \mathbf{T}^{-1} \cdot \mathbf{y}_0 \quad (17.5.13)$$

We will be interested in the *stable* solutions, that is, those that decay as $x \rightarrow \infty$. (This notion can be made more rigorous by considering *Liapunov stability*, the idea that if $\mathbf{y}_0$ is small then $\mathbf{y}$ is small for all $x > 0$.) From equation (17.5.13) we see that the criterion for stable solutions is

$$\text{Re } \lambda_i > 0 \quad i = 0, \dots, N-1 \quad (17.5.14)$$

What if the matrix $\mathbf{C}$ in equation (17.5.9) cannot be diagonalized? Then it can always be transformed to so-called Jordan canonical form, which is the “closest” it can come to being made diagonal. Using this form, one can show that criterion (17.5.14) is still the stability criterion [2].

Now consider solving equation (17.5.9) by explicit differencing as in equation (17.5.6):

$$\mathbf{y}_{n+1} = (\mathbf{I} - \mathbf{C}h) \cdot \mathbf{y}_n \quad (17.5.15)$$

and so

$$\mathbf{y}_n = (\mathbf{I} - \mathbf{C}h)^n \cdot \mathbf{y}_0 \quad (17.5.16)$$

If $\mathbf{C}$ can be diagonalized, it has a complete set of eigenvectors $\{\xi_i\}$ that can be used as a basis to expand $\mathbf{y}_0$:

$$\mathbf{y}_0 = \sum_{i=0}^{N-1} \alpha_i \xi_i \quad (17.5.17)$$

Substituting this expansion in equation (17.5.16) gives

$$\mathbf{y}_n = \sum_{i=0}^{N-1} \alpha_i (1 - h\lambda_i)^n \xi_i \quad (17.5.18)$$

If the original differential equation is stable, we require the difference scheme to be stable in that it must have bounded solutions, that is, $\mathbf{y}_n$ must be bounded as $n \rightarrow \infty$. From equation (17.5.18) we see that stability of the difference scheme requires

$$|1 - h\lambda_i| < 1 \quad i = 0, \dots, N-1 \quad (17.5.19)$$

If the λ_i are all real, then since they are positive for the differential equation to be stable, the stability criterion for the difference scheme is

$$h < \frac{2}{\lambda_{\max}} \quad (17.5.20)$$

where $\lambda_{\max}$ is the largest eigenvalue of $\mathbf{C}$.

As usual, if $\mathbf{C}$ cannot be diagonalized and so does not have a complete set of eigenvectors, then by working with the Jordan canonical form one can show the same result.

Consider now implicit differencing, which gives

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{y}'_{n+1} \quad (17.5.21)$$

or

$$\mathbf{y}_{n+1} = (\mathbf{1} + \mathbf{C}h)^{-1} \cdot \mathbf{y}_n \quad (17.5.22)$$

Criterion (17.5.19) becomes

$$|1 + h\lambda_i|^{-1} < 1 \quad i = 0, \dots, N-1 \quad (17.5.23)$$

which is satisfied for all h — the method is stable for all stepsizes. The penalty we pay for this stability is that we are required to invert a matrix at each step.

Not all equations are linear with constant coefficients, unfortunately! For the system

$$\mathbf{y}' = \mathbf{f}(\mathbf{y}) \quad (17.5.24)$$

implicit differencing gives

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(\mathbf{y}_{n+1}) \quad (17.5.25)$$

In general this is some nasty set of nonlinear equations that has to be solved iteratively at each step. Suppose we try linearizing the equations, as in Newton's method:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \left[\mathbf{f}(\mathbf{y}_n) + \frac{\partial \mathbf{f}}{\partial \mathbf{y}} \bigg|_{\mathbf{y}_n} \cdot (\mathbf{y}_{n+1} - \mathbf{y}_n) \right] \quad (17.5.26)$$

Here $\partial \mathbf{f} / \partial \mathbf{y}$ is the matrix of the partial derivatives of the right-hand side (the Jacobian matrix). Rearrange equation (17.5.26) into the form

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \left[\mathbf{1} - h \frac{\partial \mathbf{f}}{\partial \mathbf{y}} \right]^{-1} \cdot \mathbf{f}(\mathbf{y}_n) \quad (17.5.27)$$

If h is not too big, only one iteration of Newton's method may be accurate enough to solve equation (17.5.25) using equation (17.5.27). In other words, at each step we have to invert the matrix

$$\mathbf{1} - h \frac{\partial \mathbf{f}}{\partial \mathbf{y}} \quad (17.5.28)$$

to find $\mathbf{y}_{n+1}$. Solving implicit methods by linearization is called a “semi-implicit” method, so equation (17.5.27) is the *semi-implicit Euler method*. It is not guaranteed to be stable, but it usually is, because the behavior is locally similar to the case of a constant matrix $\mathbf{C}$ described above.

So far we have dealt only with implicit methods that are first-order accurate. While these are very robust, most problems will benefit from higher-order methods. There are three important classes of higher-order methods for stiff systems:

- Generalizations of the Runge-Kutta method. These consist of implicit methods, where nonlinear equations are solved by Newton iteration at each step, and semi-implicit methods that solve linear equations analogous to (17.5.27). These semi-implicit methods are often called Rosenbrock methods. The first good implementation of these ideas was by Kaps and Rentrop, and so these methods are also called Kaps-Rentrop methods.
- Generalizations of the Bulirsch-Stoer method, which extrapolate a semi-implicit sequence of integrations to zero stepsize.
- Predictor-corrector methods, most of which are descendants of Gear's backward differentiation method.

We shall give implementations of Rosenbrock and extrapolation methods. An example of a good implicit Runge-Kutta code in Fortran is RADAU [3], while several stiff predictor-corrector-type Fortran codes (LSODE, DEBDF, VODE and MEBDF) are available from Netlib [5].

Here is an important point: *It is absolutely crucial to scale your variables properly when integrating stiff problems with automatic stepsize adjustment.* As in our nonstiff routines, you will be asked to supply absolute and relative tolerances `atol` and `rtol`. In stiff problems, there are often strongly decreasing pieces of the solution that you are not particularly interested in following once they are small. Thus you should almost never integrate with a pure relative error criterion by setting `atol = 0`. A good default choice is `atol = rtol`, or sometimes a few orders of magnitude smaller.

One final warning: Solving stiff problems can sometimes lead to catastrophic precision loss. Double precision is often a requirement, not an option.

17.5.1 Rosenbrock Methods

These methods have the advantage of being relatively simple to understand and implement. For moderate accuracies (tolerances of order 10^{-4} – 10^{-5}) and moderate-sized systems ($N \lesssim 10$), they are competitive with the more complicated algorithms. For more stringent parameters, Rosenbrock methods remain reliable; they merely become less efficient than competitors like the semi-implicit extrapolation method (see below).

A Rosenbrock method seeks a solution of the form

$$\mathbf{y}(x_0 + h) = \mathbf{y}_0 + \sum_{i=1}^s b_i \mathbf{k}_i \quad (17.5.29)$$

where the corrections $\mathbf{k}_i$ are found by solving s linear equations that generalize the structure in (17.5.27):

$$(\mathbf{I} - \gamma h \mathbf{f}') \cdot \mathbf{k}_i = h \mathbf{f} \left(\mathbf{y}_0 + \sum_{j=1}^{i-1} \alpha_{ij} \mathbf{k}_j \right) + h \mathbf{f}' \cdot \sum_{j=1}^{i-1} \gamma_{ij} \mathbf{k}_j, \quad i = 1, \dots, s \quad (17.5.30)$$

Here we denote the Jacobian matrix by $\mathbf{f}'$. The coefficients γ , b_i , α_{ij} , and γ_{ij} are fixed constants independent of the problem. If $\gamma = \gamma_{ij} = 0$, this is simply a Runge-Kutta scheme. Equations (17.5.30) can be solved successively for $\mathbf{k}_1, \mathbf{k}_2, \dots$.

To minimize the matrix-vector multiplications on the right-hand side of (17.5.30), rewrite the equations in terms of quantities

$$\mathbf{g}_i = \sum_{j=1}^{i-1} \gamma_{ij} \mathbf{k}_j + \gamma \mathbf{k}_i \quad (17.5.31)$$

The equations then take the form (for four stages as an example)

$$\begin{aligned} (\mathbf{I} / \gamma h - \mathbf{f}') \cdot \mathbf{g}_1 &= \mathbf{f}(\mathbf{y}_0) \\ (\mathbf{I} / \gamma h - \mathbf{f}') \cdot \mathbf{g}_2 &= \mathbf{f}(\mathbf{y}_0 + a_{21} \mathbf{g}_1) + c_{21} \mathbf{g}_1 / h \\ (\mathbf{I} / \gamma h - \mathbf{f}') \cdot \mathbf{g}_3 &= \mathbf{f}(\mathbf{y}_0 + a_{31} \mathbf{g}_1 + a_{32} \mathbf{g}_2) + (c_{31} \mathbf{g}_1 + c_{32} \mathbf{g}_2) / h \\ (\mathbf{I} / \gamma h - \mathbf{f}') \cdot \mathbf{g}_4 &= \mathbf{f}(\mathbf{y}_0 + a_{41} \mathbf{g}_1 + a_{42} \mathbf{g}_2 + a_{43} \mathbf{g}_3) + (c_{41} \mathbf{g}_1 + c_{42} \mathbf{g}_2 + c_{43} \mathbf{g}_3) / h \end{aligned} \quad (17.5.32)$$

Here a_{ij} and c_{ij} can be expressed in terms of α_{ij} and γ_{ij} .

Note that systems where the right-hand side $\mathbf{f}(\mathbf{y}, x)$ depends explicitly on x can be handled by adding x to the list of dependent variables so that the system to be solved is

$$\begin{pmatrix} \mathbf{y} \\ x \end{pmatrix}' = \begin{pmatrix} \mathbf{f} \\ 1 \end{pmatrix} \quad (17.5.33)$$

In the routine given below, we have explicitly carried out this replacement for you, so the routines can handle right-hand sides of the form $\mathbf{f}(\mathbf{y}, x)$ without any special effort on your part.

Crucial to the success of a stiff integration scheme is an automatic stepsize adjustment algorithm. Kaps and Rentrop [6] discovered an *embedded* or Runge-Kutta-Fehlberg method as described in §17.2: Two estimates of the form (17.5.29) are computed, the “real” one $\mathbf{y}$ and a lower-order estimate $\hat{\mathbf{y}}$ with different coefficients $\hat{b}_i, i = 1, \dots, \hat{s}$, where $\hat{s} < s$ but the $\mathbf{k}_i$ are the same. The difference between $\mathbf{y}$ and $\hat{\mathbf{y}}$ leads to an estimate of the local truncation error, which can then be used for stepsize control. Kaps and Rentrop showed that the smallest value of s for which embedding is possible is $s = 4, \hat{s} = 3$, leading to a fourth-order method. By a suitable choice of parameters, only three function evaluations are needed for the four stages in each step.

In recent years, Kaps-Rentrop has lost favor to so-called *stiffly stable* methods, an implementation of which we give here as the routine `StepperRoss` (Rosenbrock Stiffly Stable), based on the Fortran routine `RODAS` [3]. It is also a fourth-order method with a third-order embedded method for stepsize control. Despite having six stages with six function evaluations, the enhanced stability makes it significantly more efficient than the Kaps-Rentrop method. Moreover, it has a simple dense output feature.

As with the earlier stepper routines in this chapter, you have to provide a functor for `derivs`, the right-hand side routine. In the structure you now must also supply a function called `jacobian` that returns $\mathbf{f}'$ and $\partial\mathbf{f}/\partial x$ as functions of x and $\mathbf{y}$. If x does not occur explicitly on the right-hand side, then `dfdx` will be zero. Usually the Jacobian matrix will be available to you by analytic differentiation of the right-hand side $\mathbf{f}$. If not, your routine will have to compute it by numerical differencing with appropriate increments $\Delta\mathbf{y}$. We will give an example of a complete derivative and jacobian structure at the end of this subsection.

The class `StepperRoss` uses a set of constants, which are provided by deriving the class from a class `Ross_constants`. This latter class is listed in a Webnote [4]. Here is the declaration of `StepperRoss`:

`stepperross.h`

```
template <class D>
struct StepperRoss : StepperBase, Ross_constants {
    Fourth-order stiffly stable Rosenbrock step for integrating stiff ODEs, with monitoring of local
    truncation error to adjust stepsize.

    typedef D Dtype;                                Make the type of derivs available to odeint.
    MatDoub dfdy;                                    f'
    VecDoub dfdx;                                    ∂f/∂x
    VecDoub k1,k2,k3,k4,k5,k6;
    VecDoub cont1,cont2,cont3,cont4;
    MatDoub a;
    StepperRoss(VecDoub_I0 &yy, VecDoub_I0 &dydxx, Doub &xx, const Doub atol1,
                const Doub rtol1, bool dens);
    void step(const Doub htry,D &derivs);
    void dy(const Doub h,D &derivs);
    void prepare_dense(const Doub h,VecDoub_I &dydxnew);
    Doub dense_out(const Int i, const Doub x, const Doub h);
    Doub error();
    struct Controller {
        Doub hnext;
        bool reject;
        bool first_step;                            first_step, errold, and hold are used by
        Doub errold;                                the predictive controller.
        Doub hold;
        Controller();
        bool success(Doub err, Doub &h);
    };
    Controller con;
};
```

The implementation will seem very familiar if you’ve looked at `StepperDopr5`, the explicit Runge-Kutta routine. Note that in the algorithm routine `dy` of `StepperRoss`, the linear equations (17.5.32) are solved by first computing the *LU* decomposition of the matrix

$1/\gamma h - \mathbf{f}'$ using the routine LUdcmp. Then the six $\mathbf{g}_i$ are found by backsubstitution of the six different right-hand sides using the routine solve in LUdcmp. Thus each step of the integration requires one call to jacobian and six calls to derivs (one call outside dy and five calls inside). The evaluation of the Jacobian matrix is roughly equivalent to N evaluations of the right-hand side $\mathbf{f}$ (although it can often be less than this, especially if commonality of code can be exploited). Thus this scheme involves about $N + 6$ function evaluations per step. Note that if N is large and the Jacobian matrix is sparse, you should replace the LU decomposition by a suitable sparse matrix procedure.

```
template <class D>
StepperRoss<D>::StepperRoss(VecDoub_IO &yy, VecDoub_IO &dydxx, Doub &xx,
    const Doub atoll,const Doub rtoll, bool dens) :
    StepperBase(yy,dydxx,xx,atoll,rtoll,dens),dfdy(n,n),dfdx(n),k1(n),k2(n),
    k3(n),k4(n),k5(n),k6(n),cont1(n),cont2(n),cont3(n),cont4(n),a(n,n)) {
    Input to the constructor are the dependent variable y[0..n-1] and its derivative dydx[0..n-1]
    at the starting value of the independent variable x. Also input are the absolute and relative
    tolerances, atol and rtol, and the boolean dense, which is true if dense output is required.
    EPS=numeric_limits<Doub>::epsilon();
}

template <class D>
void StepperRoss<D>::step(const Doub htry,D &derivs) {
    Attempts a step with stepsize htry. On output, y and x are replaced by their new values, hdid
    is the stepsize that was actually accomplished, and hnext is the estimated next stepsize.
    VecDoub dydxnew(n);
    Doub h=htry;
    derivs.jacobian(x,y,dfdx,dfdy);
    for (;;) {
        dy(h,derivs);
        Doub err=error();
        if (con.success(err,h)) break;
        if (abs(h) <= abs(x)*EPS)
            throw("stepsize underflow in StepperRoss");
    }
    derivs(x+h,yout,dydxnew);
    if (dense)
        prepare_dense(h,dydxnew);
    dydx=dydxnew;
    y=yout;
    xold=x;
    x += (hdid=h);
    hnext=con.hnext;
}

template<class D>
void StepperRoss<D>::dy(const Doub h,D &derivs) {
    Given values for n variables y[0..n-1] and their derivatives dydx[0..n-1] known at x, use the
    fourth-order stiffly stable Rosenbrock method to advance the solution over an interval h and
    store the incremented variables in yout[0..n-1]. Also store an estimate of the local truncation
    error in yerr using the embedded third-order method.
    VecDoub ytemp(n),dydxnew(n);
    Int i;
    for (i=0;i<n;i++) {
        for (Int j=0;j<n;j++) a[i][j] = -dfdy[i][j];
        a[i][i] += 1.0/(gam*h);
    }
    LUdcmp alu(a);
    for (i=0;i<n;i++)
        ytemp[i]=dydx[i]+h*d1*dfdx[i];
    alu.solve(ytemp,k1);
    for (i=0;i<n;i++)
        ytemp[i]=y[i]+a21*k1[i];
    derivs(x+c2*h,ytemp,dydxnew);
    for (i=0;i<n;i++)
        ytemp[i]=dydxnew[i]+h*d2*dfdx[i]+c21*k1[i]/h;
```

stepperross.h

Set stepsize to the initial trial value.

Compute the Jacobian and $\partial \mathbf{f} / \partial \mathbf{x}$.

Take a step.

Evaluate accuracy.

Step rejected. Try again with reduced h set by controller.

Step succeeded.

Compute coefficients for dense output.

Reuse last derivative evaluation for next step.

Used for dense output.

 LU decomposition of the matrix.Set up right-hand side for $\mathbf{g}_1$.Solve for $\mathbf{g}_1$.

Compute intermediate values of y.

Compute dydx at the intermediate values.

Set up right-hand side for $\mathbf{g}_2$.

```

alu.solve(ytemp,k2);
for (i=0;i<n;i++)
    ytemp[i]=y[i]+a31*k1[i]+a32*k2[i];
derivs(x+c3*h,ytemp,dydxnew);
for (i=0;i<n;i++)
    ytemp[i]=dydxnew[i]+h*d3*dfdx[i]+(c31*k1[i]+c32*k2[i])/h;
alu.solve(ytemp,k3);
for (i=0;i<n;i++)
    ytemp[i]=y[i]+a41*k1[i]+a42*k2[i]+a43*k3[i];
derivs(x+c4*h,ytemp,dydxnew);
for (i=0;i<n;i++)
    ytemp[i]=dydxnew[i]+h*d4*dfdx[i]+(c41*k1[i]+c42*k2[i]+c43*k3[i])/h;
alu.solve(ytemp,k4);
for (i=0;i<n;i++)
    ytemp[i]=y[i]+a51*k1[i]+a52*k2[i]+a53*k3[i]+a54*k4[i];
Doub xph=x+h;
derivs(xph,ytemp,dydxnew);
for (i=0;i<n;i++)
    k6[i]=dydxnew[i]+(c51*k1[i]+c52*k2[i]+c53*k3[i]+c54*k4[i])/h;
alu.solve(k6,k5);
for (i=0;i<n;i++)
    ytemp[i] += k5[i];
derivs(xph,ytemp,dydxnew);
for (i=0;i<n;i++)
    k6[i]=dydxnew[i]+(c61*k1[i]+c62*k2[i]+c63*k3[i]+c64*k4[i]+c65*k5[i])/h;
alu.solve(k6,yerr);
for (i=0;i<n;i++)
    yout[i]=ytemp[i]+yerr[i];
}
template <class D>
void StepperRoss<D>::prepare_dense(const Doub h,VecDoub_I &dydxnew) {
Store coefficients of interpolating polynomial for dense output in cont1...cont4.
    for (Int i=0;i<n;i++) {
        cont1[i]=y[i];
        cont2[i]=yout[i];
        cont3[i]=d21*k1[i]+d22*k2[i]+d23*k3[i]+d24*k4[i]+d25*k5[i];
        cont4[i]=d31*k1[i]+d32*k2[i]+d33*k3[i]+d34*k4[i]+d35*k5[i];
    }
}
template <class D>
Doub StepperRoss<D>::dense_out(const Int i,const Doub x,const Doub h) {
Evaluate interpolating polynomial for y[i] at location x, where xold ≤ x ≤ xold + h.
    Doub s=(x-xold)/h;
    Doub s1=1.0-s;
    return cont1[i]*s1+s*(cont2[i]+s1*(cont3[i]+s*cont4[i]));
}
template <class D>
Doub StepperRoss<D>::error() {
Use yerr to compute norm of scaled error estimate. A value less than one means the step was
successful.
    Doub err=0.0,sk;
    for (Int i=0;i<n;i++) {
        sk=atol+rtol*MAX(abs(y[i]),abs(yout[i]));
        err += SQR(yerr[i]/sk);
    }
    return sqrt(err/n);
}

```

Stepsize control depends on the fact that

$$\begin{aligned}
 y_{\text{exact}} &= y + O(h^5) \\
 y_{\text{exact}} &= \hat{y} + O(h^4)
 \end{aligned}
 \tag{17.5.34}$$

Thus

$$|y - \hat{y}| = O(h^4) \quad (17.5.35)$$

Referring back to the steps leading from equation (17.2.4) to equation (17.2.12), we see that the new stepsize should be chosen as in equation (17.2.12) but with the exponent 1/5 replaced by 1/4. Also, experience shows that it is wise to prevent too large a stepsize change in one step, otherwise we will probably have to undo the large change in the next step. We adopt 0.2 and 6 as the maximum allowed decrease and increase of h in one step.

Methods for integrating stiff equations do not suffer from the stability limitations that led to the PI controller of §17.2.1. However, stiff problems often need a rapid decrease in stepsize even when the previous step is successful. Also, sometimes the effective order of the method can be lower than the simple Taylor series prediction. Gustafsson [7] has proposed a *predictive controller* that does a good job of dealing with these problems. The resulting formula is

$$h_{n+1} = Sh_n \left(\frac{1}{\text{err}_n} \right)^{1/4} \frac{h_n}{h_{n-1}} \left(\frac{\text{err}_{n-1}}{\text{err}_n} \right)^{1/4} \quad (17.5.36)$$

It is used only when a step is accepted.

```
template <class D>
StepperRoss<D>::Controller::Controller() : reject(false), first_step(true) {}
Step size controller for fourth-order Rosenbrock method.
template <class D>
bool StepperRoss<D>::Controller::success(Doub err, Doub &h) {
Returns true if err ≤ 1, false otherwise. If step was successful, sets hnext to the estimated
optimal stepsize for the next step. If the step failed, reduces h appropriately for another try.
    static const Doub safe=0.9, fac1=5.0, fac2=1.0/6.0;
    Doub fac=MAX(fac2, MIN(fac1, pow(err, 0.25)/safe));
    Doub hnew=h/fac;
    if (err <= 1.0) {
        if (!first_step) {
            Doub facpred=(hold/h)*pow(err*err/errold, 0.25)/safe;
            facpred=MAX(fac2, MIN(fac1, facpred));
            fac=MAX(fac, facpred);
            hnew=h/fac;
        }
        first_step=false;
        hold=h;
        errold=MAX(0.01, err);
        if (reject)
            hnew=(h >= 0.0 ? MIN(hnew, h) : MAX(hnew, h));
        hnext=hnew;
        reject=false;
        return true;
    } else {
        h=hnew;
        reject=true;
        return false;
    }
}
```

stepperross.h

Ensure $1/\text{fac1} \leq h_{\text{new}}/h \leq 1/\text{fac2}$.
Step succeeded.
Predictive control.
Don't let step increase if last one was rejected.
Truncation error too large, reduce stepsize.

As an example of how StepperRoss is used, one can solve the system

$$\begin{aligned} y_0' &= -0.013y_0 - 1000y_0y_2 \\ y_1' &= -2500y_1y_2 \\ y_2' &= -0.013y_0 - 1000y_0y_2 - 2500y_1y_2 \end{aligned} \quad (17.5.37)$$

with initial conditions

$$y_0(0) = 1, \quad y_1(0) = 1, \quad y_2(0) = 0 \quad (17.5.38)$$

(This is test problem D4 in [8].) We integrate the system up to $x = 50$ with an initial stepsize of $h = 2.9 \times 10^{-4}$ using `Odeint`. We set $\text{atol} = \text{rtol} = 10^{-5}$. The right-hand side routine for this problem is given below. Even though the ratio of largest to smallest decay constants for this problem is around 10^6 , `StepperRoss` succeeds in integrating this set in only 11 steps with 67 function evaluations. By contrast, the explicit Runge-Kutta routine `StepperDopr5` requires almost 60,000 steps and over 400,000 function evaluations!

```
struct rhs {
    void operator() (const Doub x, VecDoub_I &y, VecDoub_O &dydx) {
        dydx[0] = -0.013*y[0]-1000.0*y[0]*y[2];
        dydx[1] = -2500.0*y[1]*y[2];
        dydx[2] = -0.013*y[0]-1000.0*y[0]*y[2]-2500.0*y[1]*y[2];
    }
    void jacobian(const Doub x, VecDoub_I &y, VecDoub_O &dxdy,
        MatDoub_O &d2dy) {
        Int n=y.size();
        for (Int i=0;i<n;i++) d2dy[i]=0.0;
        dxdy[0][0] = -0.013-1000.0*y[2];
        dxdy[0][1] = 0.0;
        dxdy[0][2] = -1000.0*y[0];
        dxdy[1][0] = 0.0;
        dxdy[1][1] = -2500.0*y[2];
        dxdy[1][2] = -2500.0*y[1];
        dxdy[2][0] = -0.013-1000.0*y[2];
        dxdy[2][1] = -2500.0*y[2];
        dxdy[2][2] = -1000.0*y[0]-2500.0*y[1];
    }
};
```

17.5.2 Semi-Implicit Extrapolation Method

The Bulirsch-Stoer method, which discretizes the differential equation using the modified midpoint rule, does not work for stiff problems. For many years, successful extrapolation-type routines for stiff equations were based on an algorithm of Bader and Deuflhard [9]. This algorithm uses a semi-implicit version of the midpoint method that has an even error series.

Not long afterward, however, Deuflhard [10] investigated a semi-implicit version of the Euler method, equation (17.5.27). This does not have an even error series. Nevertheless, it turns out that for high precision, using this method as the basis of an extrapolation scheme is even more efficient than using the semi-implicit midpoint rule. (Some theoretical insight into this behavior is provided in §VI.5 of [3].) Since `StepperRoss` is generally satisfactory for low precision, this method is a good companion. We give it as `StepperSie` (“Semi-Implicit Euler”).

The basic equation of the method is equation (17.5.27) rewritten in the form

$$\left[1/h - \frac{\partial \mathbf{f}}{\partial \mathbf{y}}\right] \cdot (\mathbf{y}_{n+1} - \mathbf{y}_n) = \mathbf{f}(\mathbf{y}_n) \quad (17.5.39)$$

A sequence of stepsizes $h_i = H/n_i$ is used with this equation to advance the solution a distance H . The linear equations are solved with LU decomposition. Polynomial extrapolation is used as in the original Bulirsch-Stoer method, except that in equation (17.3.8) the ratio of stepsizes is not squared because the error series is not even.

Instead of making the replacement (17.5.33) in the above formula, it turns out to be slightly better to add a single simplified Newton iteration of the fully implicit Euler step (17.5.25):

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(x_{n+1}, \mathbf{y}_{n+1}) \longrightarrow \left[1 - h \frac{\partial \mathbf{f}}{\partial \mathbf{y}}\right] \cdot (\mathbf{y}_{n+1} - \mathbf{y}_n) = h\mathbf{f}(x_{n+1}, \mathbf{y}_n) \quad (17.5.40)$$

This costs an extra function evaluation but avoids the computation of $\partial \mathbf{f} / \partial x$. In the code, we leave $\partial \mathbf{f} / \partial x$ as an argument of the jacobian function for compatibility with `StepperRoss`, but it is not used.

Another difference from `StepperRoss` is that the Jacobian does not have to be exact. Its main role is to ensure stability, not accuracy. Accordingly, the code has a test to see when the Jacobian needs to be recomputed.

Differences from `StepperBS` include

- The default stepsize sequence is

$$n = 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64, \dots, [n_j = 2n_{j-2}], \dots \quad (17.5.41)$$

- The work per unit step now includes the cost of Jacobian evaluations as well as function evaluations. We count one Jacobian evaluation as equivalent to five function evaluations by default, but it could be as large as N , the number of equations. The work per unit step also includes the cost of the LU decomposition and the backsubstitutions, each set by default to the cost of a function evaluation.
- Several checks for instability are included. If the estimated error err_k starts increasing with k during a step, the step is restarted with the stepsize reduced by a factor of two. Similarly, a stability test is made for $k = 0, 1$ during the Euler step and the step is rejected if the test is failed. You could add a test for failure of the LU decomposition and similarly reduce the stepsize if that happened.

The routine, which is based on the Fortran routine `SEULEX` [3], next follows.

17.5.3 Implementation of Semi-Implicit Extrapolation Method

The routine `StepperSie` is an excellent routine for all stiff problems, competitive with the best Gear-type routines. `StepperRoss` is often better in execution time for moderate N and $\epsilon \lesssim 10^{-5}$. The detailed implementation is listed in a Webnote [11].

CITED REFERENCES AND FURTHER READING:

- Gear, C.W. 1971, *Numerical Initial Value Problems in Ordinary Differential Equations* (Englewood Cliffs, NJ: Prentice-Hall).[1]
- Hairer, E., Nørsett, S.P., and Wanner, G. 1993, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed. (New York: Springer). Fortran codes at <http://www.unige.ch/~hairer/software.html>.[2]
- Hairer, E., Nørsett, S.P., and Wanner, G. 1996, *Solving Ordinary Differential Equations II. Stiff and Differential-Algebraic Problems*, 2nd ed. (New York: Springer).[3]
- Numerical Recipes Software 2007, "Constants for Stiffly Stable Rosenbrock Method," *Numerical Recipes Webnote No. 23*, at <http://www.nr.com/webnotes?23> [4]
- Netlib: <http://www.netlib.org/>.[5]
- Kaps, P., and Rentrop, P. 1979, "Generalized Runge-Kutta Methods of Order Four with Stepsize Control for Stiff Ordinary Differential Equations," *Numerische Mathematik*, vol. 33, pp. 55–68.[6]
- Gustafsson, K. 1994, "Control Theoretic Techniques for Stepsize Selection in Implicit Runge-Kutta Methods," *ACM Transactions on Mathematical Software*, vol. 20, pp. 496–517.[7]
- Enright, W.H., and Pryce, J.D. 1987, "Two FORTRAN Packages for Assessing Initial Value Methods," *ACM Transactions on Mathematical Software*, vol. 13, pp. 1–27.[8]
- Bader, G., and Deuflhard, P. 1983, "A Semi-Implicit Mid-Point Rule for Stiff Systems of Ordinary Differential Equations," *Numerische Mathematik*, vol. 41, pp. 373–398.[9]
- Deuflhard, P. 1985, "Recent Progress in Extrapolation Methods for Ordinary Differential Equations," *SIAM Review*, vol. 27, pp. 505–535.[10]
- Numerical Recipes Software 2007, "StepperSie Implementation," *Numerical Recipes Webnote No. 24*, at <http://www.nr.com/webnotes?24> [11]